Informe Triggers Módulo 3

Telemetría e IoT con Operación Edge

Versión: 1.0

28 de abril de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TRG-M03-01 — trg_telemetria_inmutable

Descripción	Impide cualquier modificación o eliminación sobre registros de telemetría ya persistidos. Una vez que una medición queda almacenada en la base de datos principal, su contenido es permanente e inalterable. Este trigger actúa como el último guardián de la integridad histórica del dato, garantizando que ni procesos administrativos, ni migraciones, ni errores de código puedan vulnerar el principio de inmutabilidad que exige RF-53 y que es fundamental para la validez de los registros contables bajo NIC 41.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Restricciones: *"Una vez que una medición ha sido almacenada y validada, el sistema no permitirá su modificación ni eliminación. El historial debe ser completamente inalterable."*
Justificación	La inmutabilidad de telemetría es un principio de integridad de datos, no lógica de negocio. Debe aplicarse a nivel de motor de base de datos para garantizar que ningún proceso externo (incluyendo procesos de mantenimiento, migraciones o bugs) pueda vulnerarla.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_telemetria_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            'Los registros de telemetría son inmutables. No se
permite eliminar lecturas almacenadas. RF-53.'
            USING ERRCODE = 'P0001';
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
```

```
            'Los registros de telemetría son inmutables. No se
permite modificar lecturas almacenadas. RF-53.'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_telemetria_inmutable
BEFORE UPDATE OR DELETE ON modulo3.telemetrias
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_telemetria_inmutable();
```

TRG-M03-02 — trg_telemetria_timestamp_no_futuro

Descripción	Valida que los timestamps de captura y envío de una lectura de telemetría no sean posteriores al momento actual del servidor. Un dato con marca de tiempo futura es cronológicamente imposible y representa una desincronización del reloj del dispositivo IoT. Este trigger protege la coherencia de las series temporales del sistema, evitando que registros con timestamps incorrectos contaminen el historial que alimenta los modelos predictivos de M04 y la valoración financiera de M06.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Restricciones: *"No se permitirá el registro de datos cuya marca de tiempo sea posterior a la hora actual del servidor."* / Proceso Fase 3: *"Si timestamp_captura > hora del servidor → Se clasifica como ERROR_TIEMPO → Se rechaza inmediatamente el dato."*
Justificación	Es una regla de integridad temporal que debe garantizarse independientemente del canal por el que llegue el dato (API, sincronización de buffer, procesos batch).

	La BD es el último guardián.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_telemetria_timestamp_no_futuro()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.timestamp_captura > NOW() THEN
        RAISE EXCEPTION
            'El timestamp_captura [%] es posterior a la hora actual del
servidor. Dato rechazado. (ERROR_TIEMPO -- RF-53 Fase 3)',
            NEW.timestamp_captura
            USING ERRCODE = 'P0001';
    END IF;
    IF NEW.timestamp_envio > NOW() THEN
        RAISE EXCEPTION
            'El timestamp_envio [%] es posterior a la hora actual del
servidor. Dato rechazado. (RF-53)',
            NEW.timestamp_envio
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_telemetria_timestamp_no_futuro
BEFORE INSERT ON modulo3.telemetrias
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_telemetria_timestamp_no_futuro();
```

TRG-M03-03 — trg_telemetria_estado_calidad_valido

Descripción	Garantiza que únicamente los estados de calidad que deben persistirse en la base de datos principal sean insertados en la tabla de telemetrías. Según RF-53, solo LECTURA_VALIDA, FUERA_DE_RANGO y ERROR_CALIBRACION tienen representación física en BD; los demás estados como ERROR_ESTRUCTURA, ERROR_AUTENTICACION,

	ERROR_DUPLICADO, ERROR_TIEMPO, ERROR_UNIDAD y ERROR_SENSOR se registran exclusivamente en bitácora. Este trigger evita que procesos incorrectos persistan datos que debieron ser rechazados antes de llegar a la capa de almacenamiento principal.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Salida: *"No se permite el uso de estados adicionales, alias, códigos internos o valores distintos al catálogo oficial definido."* / CA-8: *"El sistema debe rechazar cualquier estado que no pertenezca explícitamente al catálogo definido."*
Justificación	La BD puede usar el tipo ENUM ya definido enum_telemetria_estado_calidad, pero adicionalmente debe validar que solo los estados que corresponden a persistencia en base principal sean insertados (los estados ERROR_* que no se persisten deben ser bloqueados a nivel de BD). Esto protege contra inserciones incorrectas desde procesos batch o migraciones.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_telemetria_estado_calidad_valido()
RETURNS TRIGGER AS $$
BEGIN
    -- Según RF-53 Fase 9: Solo se persisten en BD principal:
    -- LECTURA_VALIDA, FUERA_DE_RANGO, ERROR_CALIBRACION
    -- Los demás estados se registran ÚNICAMENTE en bitácora.
    IF NEW.estado_calidad NOT IN (
        'LECTURA_VALIDA',
        'FUERA_DE_RANGO',
        'ERROR_CALIBRACION'
    ) THEN
        RAISE EXCEPTION
            'El estado_calidad [%] no puede persistirse en la base de
datos principal. '
            'Solo LECTURA_VALIDA, FUERA_DE_RANGO y ERROR_CALIBRACION son
persistibles. '
            'Los demás estados van únicamente a bitácora. (RF-53 Fase
9)',
```

```
            NEW.estado_calidad
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_telemetria_estado_calidad_valido
BEFORE INSERT ON modulo3.telemetrias
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_telemetria_estado_calidad_valido();
```

TRG-M03-04 — trg_telemetria_calibracion_coherencia

Descripción	Valida la coherencia entre el campo booleano calibrado y el campo valor_ajustado. Si un registro declara que fue calibrado (calibrado = true), obligatoriamente debe tener un valor_ajustado no nulo, ya que la calibración produce precisamente ese valor transformado. La situación contraria, declarar calibración sin resultado de calibración, representa una inconsistencia estructural del dato que podría inducir errores en los modelos de IA y en la valoración contable que consumen el dato calibrado como referencia.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Fase 7: *"Si NO existen parámetros: valor_ajustado = null, calibrado = false, estado = ERROR_CALIBRACION."* / *"El flag 'calibrado' debe ser consistente con el campo estructural calibrado."*
Justificación	La coherencia entre calibrado (boolean) y valor_ajustado es una regla de integridad referencial entre campos propios de la misma fila, adecuada para BD.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_telemetria_calibracion_coherencia()
RETURNS TRIGGER AS $$
BEGIN
    -- Si calibrado = true, valor_ajustado NO puede ser nulo
    IF NEW.calibrado = TRUE AND NEW.valor_ajustado IS NULL THEN
        RAISE EXCEPTION
            'Incoherencia: calibrado = true pero valor_ajustado es NULL.
'
            'Si el dato fue calibrado debe tener valor_ajustado. (RF-53
Fase 7)'
            USING ERRCODE = 'P0001';
    END IF;

    -- Si calibrado = false y estado = ERROR_CALIBRACION,
    -- el valor_ajustado debería ser nulo o igual al crudo (no
transformado)
    -- Esta validación es informativa: no bloqueamos, solo garantizamos
coherencia lógica
    -- El rechazo completo por ERROR_CALIBRACION es responsabilidad del
backend
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_telemetria_calibracion_coherencia
BEFORE INSERT ON modulo3.telemetrias
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_telemetria_calibracion_coherencia();
```

TRG-M03-05 — trg_telemetria_buffer_ventana_maxima

Descripción	Rechaza la inserción de datos provenientes de buffer local (origen = BUFFER_LOCAL) cuyo timestamp_captura sea anterior a 72 horas desde el momento actual. Este límite, definido explícitamente en RF-53 y RF-54, establece la ventana máxima aceptable para la sincronización diferida de datos acumulados durante periodos de

	desconexión. Datos más antiguos que ese umbral no pueden garantizar relevancia operativa ni coherencia con el estado actual del activo biológico, por lo que deben ser rechazados incluso si llegan al backend.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Fase 3.1: *"Los datos deben procesarse en orden ascendente de timestamp_captura."* / *"El sistema debe aceptar datos con retraso máximo de 72 horas. Datos fuera de esta ventana deben ser rechazados."*
Justificación	El límite de 72 horas para datos de buffer es una restricción de integridad temporal que aplica independientemente del origen del proceso que realiza la inserción.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_telemetria_buffer_ventana_maxima()
RETURNS TRIGGER AS $$
BEGIN
    -- Solo aplica para datos de BUFFER_LOCAL
    IF NEW.origen = 'BUFFER_LOCAL' THEN
        -- Máximo 72 horas de antigüedad en datos de buffer
        IF NEW.timestamp_captura < (NOW() - INTERVAL '72 hours') THEN
            RAISE EXCEPTION
                'Dato de BUFFER_LOCAL rechazado: timestamp_captura [%]
supera la ventana máxima '
                'de 72 horas permitida para sincronización diferida.
(RF-53 Fase 3.1)',
                NEW.timestamp_captura
                USING ERRCODE = 'P0001';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_telemetria_buffer_ventana_maxima
BEFORE INSERT ON modulo3.telemetrias
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_telemetria_buffer_ventana_maxima();
```

TRG-M03-06 — trg_telemetria_valor_valido

Descripción	Verifica que el campo valor_crudo no sea nulo en ningún registro de telemetría. El valor de la medición es el dato fundamental que justifica la existencia del registro; un registro de telemetría sin valor es structuralmente vacío e inservible para cualquier proceso analítico, de alerta o contable. Aunque el tipo NUMERIC del campo impide almacenar texto, no impide valores nulos si la restricción NOT NULL no está correctamente configurada, por lo que este trigger actúa como validación explícita con mensaje descriptivo.
Aplicado a	RF-53, RF-55.1
Restricción origen RF	RF-53 — Entradas: *"valor: Debe ser numérica, no nula y dentro del rango físico del sensor."* / RF-55.1 — Restricciones: *"Los valores registrados deben ser numéricos, positivos y representar un único dato escalar."*
Justificación	El valor numérico no nulo es una restricción de dominio sobre el dato, independiente de cualquier regla de negocio del backend.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_telemetria_valor_valido()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.valor_crudo IS NULL THEN
        RAISE EXCEPTION
            'El campo valor_crudo no puede ser nulo en un registro de
telemetría. (RF-53)'
            USING ERRCODE = 'P0001';
    END IF;
    -- Para variables que admiten negativos (temperatura ambiente puede
ser < 0)
    -- No se bloquea por signo aquí; eso es dominio del catálogo I3P-1
(backend)
    -- Solo garantizamos que el valor sea numérico (ya garantizado por
tipo NUMERIC)
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_telemetria_valor_valido
BEFORE INSERT ON modulo3.telemetrias
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_telemetria_valor_valido();
```

TRG-M03-07 — trg_alertas_telemetria_inmutable

Descripción	Impide la eliminación de cualquier registro de alerta de telemetría. Las alertas constituyen evidencia histórica del comportamiento del sistema y del estado de los activos biológicos en momentos específicos. RF-57 establece explícitamente que las alertas no deben eliminarse y solo pueden cambiar de estado a través del ciclo de vida definido. Eliminar una alerta borraría trazabilidad clínica y técnica que podría ser requerida en auditorías, análisis de patrones o procesos de mejora continua del sistema.
Aplicado a	RF-57
Restricción origen RF	RF-57 — Restricciones: *"Las alertas generadas: No deben eliminarse. No deben modificarse en su contenido histórico. Solo pueden cambiar de estado."*
Justificación	La inmutabilidad del historial de alertas es un principio de auditoría que debe ser garantizado a nivel de motor de BD.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_alertas_telemetria_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION
        'Las alertas de telemetría son inmutables. No se permite
```

```
eliminar registros de alerta. '
        'Solo puede actualizarse el estado. (RF-57 Restricciones)'
        USING ERRCODE = 'P0001';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_alertas_telemetria_inmutable
BEFORE DELETE ON modulo3.alertas_telemetria
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_alertas_telemetria_inmutable();
```

TRG-M03-08 — trg_alertas_campos_historicos_inmutables

Descripción	Protege los campos que conforman el contenido histórico de una alerta, permitiendo únicamente la modificación de los campos relacionados con la gestión del ciclo de vida como el estado, el reconocimiento y la fecha de reconocimiento. Los campos que identifican el origen de la alerta, el valor que la disparó, el sensor involucrado, la regla activada y el timestamp de creación son inmutables una vez persistidos. Esto garantiza que la evidencia de por qué y cuándo se generó la alerta permanezca intacta e inalterable para cualquier proceso posterior de auditoría o análisis.
Aplicado a	RF-57
Restricción origen RF	RF-57 — Restricciones: *"No deben modificarse en su contenido histórico. Solo pueden cambiar de estado (ej. ACTIVA → RESUELTA)."*
Justificación	El contenido histórico de la alerta (valor detectado, sensor, regla, timestamp de creación) no puede modificarse. Solo el

	estado y los campos de gestión del ciclo de vida deben ser actualizables.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_alertas_campos_historicos_inmutables()
RETURNS TRIGGER AS $$
BEGIN
    -- Campos históricos que NO pueden modificarse
    IF NEW.id_regla_alerta       <> OLD.id_regla_alerta       OR
       NEW.id_lectura_sensor     <> OLD.id_lectura_sensor     OR
       NEW.id_sensor             <> OLD.id_sensor             OR
       NEW.id_dispositivo_iot    <> OLD.id_dispositivo_iot    OR
       NEW.valor_detectado       <> OLD.valor_detectado       OR
       NEW.mensaje               <> OLD.mensaje               OR
       NEW.es_generada_edge      <> OLD.es_generada_edge      OR
       NEW.fecha_creacion        <> OLD.fecha_creacion
    THEN
        RAISE EXCEPTION
            'El contenido histórico de la alerta es inmutable. Solo se
permite '
            'actualizar el estado, reconocida_por y
fecha_reconocimiento. (RF-57 Restricciones)'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_alertas_campos_historicos_inmutables
BEFORE UPDATE ON modulo3.alertas_telemetria
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_alertas_campos_historicos_inmutables();
```

TRG-M03-09 — trg_alertas_timestamp_no_futuro

Descripción	Valida que la fecha de creación de una alerta y, si está presente, la fecha de reconocimiento, no sean posteriores al momento actual del servidor. Una alerta registrada con timestamp futuro indica un

	error de sincronización o una manipulación incorrecta del dato, y comprometería la coherencia del historial cronológico de alertas del sistema. El ordenamiento temporal correcto de las alertas es fundamental para el análisis de patrones de incidentes y para el cumplimiento de los SLA de atención definidos en RF-57.
Aplicado a	RF-57
Restricción origen RF	RF-57 — Entradas: *"timestamp_evento: Debe ser coherente y no futuro."*
Justificación	Integridad temporal del registro de alerta. Una alerta no puede ser generada con timestamp futuro.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_alertas_timestamp_no_futuro()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_creacion > NOW() THEN
        RAISE EXCEPTION
            'La fecha_creacion de la alerta [%] no puede ser futura.
(RF-57)',
            NEW.fecha_creacion
            USING ERRCODE = 'P0001';
    END IF;
    IF NEW.fecha_reconocimeinot IS NOT NULL AND NEW.fecha_reconocimeinot
> NOW() THEN
        RAISE EXCEPTION
            'La fecha_reconocimiento de la alerta [%] no puede ser
futura. (RF-57)',
            NEW.fecha_reconocimeinot
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_alertas_timestamp_no_futuro
BEFORE INSERT ON modulo3.alertas_telemetria
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_alertas_timestamp_no_futuro();
```

TRG-M03-10 — trg_alertas_reconocimiento_coherencia

Descripción	Garantiza que los campos de reconocimiento de una alerta sean mutuamente dependientes y coherentes. Si se registra quién reconoció la alerta (reconocida_por), obligatoriamente debe existir la fecha en que ocurrió ese reconocimiento (fecha_reconocimeinot), y viceversa. Permitir que uno de estos campos esté presente sin el otro crearía registros incompletos que impedirían auditar correctamente el ciclo de atención de la alerta y calcular los tiempos de respuesta exigidos por los SLA de RF-57.
Aplicado a	RF-57
Restricción origen RF	RF-57 — Proceso Fase 9: *"El usuario puede transicionar: ACTIVA → EN_ATENCION (cuando un usuario la reconoce). Cada transición queda registrada con: Timestamp, Usuario responsable."*
Justificación	Si se registra reconocida_por, la fecha_reconocimeinot debe estar presente y no ser nula. Esta coherencia entre campos es validación de dominio de BD.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_alertas_reconocimiento_coherencia()
RETURNS TRIGGER AS $$
BEGIN
    -- Si se reconoce, debe haber fecha
    IF NEW.reconocida_por IS NOT NULL AND NEW.fecha_reconocimeinot IS
NULL THEN
        RAISE EXCEPTION
            'Si reconocida_por está definido, fecha_reconocimiento es
obligatoria. (RF-57 Fase 9)'
            USING ERRCODE = 'P0001';
```

```
    END IF;
    -- Si hay fecha, debe haber responsable
    IF NEW.fecha_reconocimeinot IS NOT NULL AND NEW.reconocida_por IS
NULL THEN
        RAISE EXCEPTION
            'Si fecha_reconocimiento está definida, reconocida_por es
obligatorio. (RF-57 Fase 9)'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_alertas_reconocimiento_coherencia
BEFORE INSERT OR UPDATE ON modulo3.alertas_telemetria
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_alertas_reconocimiento_coherencia();
```

TRG-M03-11 — trg_reglas_alertas_umbral_coherencia

Descripción	Verifica que en toda regla de alerta donde se definan tanto `umbral_min` como `umbral_max`, el valor mínimo sea estrictamente menor que el valor máximo. Una regla donde el umbral mínimo sea igual o superior al máximo representa una configuración matemáticamente incoherente que haría imposible clasificar cualquier lectura como normal, generaría alertas falsas o nunca generaría alertas válidas. Este trigger protege la integridad de la configuración del sistema de alertas antes de que reglas incorrectas sean aplicadas en producción.
Aplicado a	RF-57
Restricción origen RF	RF-57 — Proceso Fase 5: *"Reglas simples (umbrales individuales por variable)."* Los umbrales deben ser coherentes: `umbral_min` < `umbral_max` cuando

	ambos están definidos.
Justificación	Integridad de dominio en la configuración de reglas. Una regla con umbral_min > umbral_max es matemáticamente incoherente e imposibilitaría la evaluación correcta.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_reglas_alertas_umbral_coherencia()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.umbral_min IS NOT NULL
       AND NEW.umbral_max IS NOT NULL
       AND NEW.umbral_min >= NEW.umbral_max THEN
        RAISE EXCEPTION
            'umbral_min [%] debe ser estrictamente menor que umbral_max
[%] en la regla de alerta. (RF-57 Fase 5)',
            NEW.umbral_min, NEW.umbral_max
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_reglas_alertas_umbral_coherencia
BEFORE INSERT OR UPDATE ON modulo3.reglas_alertas
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_reglas_alertas_umbral_coherencia();
```

TRG-M03-12 — trg_buffer_fecha_captura_no_futura

Descripción	Impide que un dato almacenado en el buffer local tenga una fecha de captura posterior al momento actual del servidor. Un dato con timestamp futuro en el buffer es imposible desde el punto de vista físico, ya que el dispositivo IoT no puede capturar datos de un momento que aún no ha ocurrido. Este tipo de registro indicaría una desincronización severa del reloj del

	dispositivo, y permitir su almacenamiento en buffer contaminaría la cola de sincronización con datos que serían rechazados posteriormente por RF-53 en cualquier caso.
Aplicado a	RF-54, RF-53
Restricción origen RF	RF-54 — Restricciones: *"Integridad e inmutabilidad de datos: Los datos almacenados no pueden ser modificados. Debe garantizarse su integridad mediante mecanismos de verificación (checksum) y la inmutabilidad del timestamp de captura."* / RF-53 Fase 3: datos con timestamp futuro son ERROR_TIEMPO.
Justificación	El buffer no puede almacenar datos con timestamp futuro, ya que esto violaría la coherencia temporal del sistema.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_buffer_fecha_captura_no_futura()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_captura > NOW() THEN
        RAISE EXCEPTION
            'La fecha_captura del dato en buffer [%] no puede ser
futura. (RF-54 / RF-53 Fase 3)',
            NEW.fecha_captura
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_buffer_fecha_captura_no_futura
BEFORE INSERT ON modulo3.buffers
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_buffer_fecha_captura_no_futura();
```

TRG-M03-13 — trg_buffer_registro_inmutable

Descripción	Protege la integridad de los datos originales almacenados en el buffer local, impidiendo cualquier modificación sobre los campos que conforman el dato tal como fue capturado por el sensor, específicamente el dispositivo origen, el sensor, el payload completo, la fecha de captura y la fecha de creación del registro en buffer. Solo los campos relacionados con el proceso de sincronización, como el estado de sincronización, la fecha de sincronización y el contador de intentos, pueden ser actualizados. Esto garantiza que el dato que eventualmente llega al backend a través de la sincronización sea exactamente el mismo que fue capturado en campo.
Aplicado a	RF-54
Restricción origen RF	RF-54 — Restricciones: *"Integridad e inmutabilidad de datos: Los datos almacenados no pueden ser modificados."* / *"Eliminación condicionada por confirmación: Los datos solo pueden eliminarse del buffer tras confirmación explícita de persistencia exitosa en el backend."*
Justificación	Los campos de trazabilidad del dato original (payload, fecha_captura, sensor, dispositivo) no pueden modificarse una vez almacenados. Solo se permite actualizar los campos de estado de sincronización.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_buffer_registro_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    -- Los campos del dato original son inmutables
    IF NEW.id_dispositivo  <> OLD.id_dispositivo  OR
       NEW.id_sensor       <> OLD.id_sensor        OR
       NEW.payload_row::text <> OLD.payload_row::text OR
       NEW.fecha_captura   <> OLD.fecha_captura    OR
       NEW.fecha_creacion  <> OLD.fecha_creacion
    THEN
        RAISE EXCEPTION
            'Los datos originales almacenados en buffer son inmutables.
```

```
'
            'Solo se permite actualizar es_sincronizado,
fecha_sincronizacion e intentos_sincronizacion. (RF-54)'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_buffer_registro_inmutable
BEFORE UPDATE ON modulo3.buffers
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_buffer_registro_inmutable();
```

TRG-M03-14 — trg_buffer_ventana_maxima_72h

Descripción	Calcula automáticamente el campo `horas_buffer` basado en la diferencia entre el momento de inserción y el `timestamp_captura` del dato, y rechaza datos cuya antigüedad supere las 72 horas. Esta ventana máxima está definida tanto en RF-54 como capacidad mínima de almacenamiento local como en RF-53 como límite de aceptación de datos diferidos. El trigger centraliza este cálculo en la BD para garantizar que el campo `horas_buffer` siempre refleje el valor correcto independientemente de cómo el dato llegó a la capa de persistencia.
Aplicado a	RF-54
Restricción origen RF	RF-54 — Precondiciones: *"El dispositivo debe contar con almacenamiento local no volátil suficiente para soportar al menos 72 horas de operación sin conectividad."* / RF-53 Fase 3.1: *"El sistema debe aceptar datos con retraso máximo de 72 horas."*
Justificación	El `horas_buffer` calculado debe ser

	coherente con el máximo permitido. Aunque la política es configurable, la BD valida la ventana máxima absoluta.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_buffer_ventana_maxima_72h()
RETURNS TRIGGER AS $$
DECLARE
    v_horas_diferencia NUMERIC;
BEGIN
    v_horas_diferencia := EXTRACT(EPOCH FROM (NOW() -
NEW.fecha_captura)) / 3600.0;

    -- El campo horas_buffer se calcula y valida
    NEW.horas_buffer := FLOOR(v_horas_diferencia);

    -- Datos con más de 72 horas no pueden almacenarse en buffer (ya son
inválidos)
    IF v_horas_diferencia > 72 THEN
        RAISE EXCEPTION
            'Dato rechazado: la antigüedad del dato [% horas] supera la
ventana máxima '
            'de 72 horas de buffer permitida. (RF-54 / RF-53 Fase 3.1)',
            ROUND(v_horas_diferencia::numeric, 2)
            USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_buffer_ventana_maxima_72h
BEFORE INSERT ON modulo3.buffers
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_buffer_ventana_maxima_72h();
```

TRG-M03-15 — trg_buffer_sincronizacion_coherencia

Descripción	Garantiza la coherencia del proceso de sincronización del buffer en dos sentidos. Primero, si un registro es marcado como

	sincronizado (es_sincronizado = true), debe registrar obligatoriamente la fecha en que ocurrió esa sincronización. Segundo, una vez que un registro ha sido confirmado como sincronizado, ese estado no puede revertirse a no sincronizado, ya que la sincronización es una transición unidireccional equivalente a la confirmación de persistencia en el backend. Revertir este estado podría provocar reenvíos duplicados del mismo dato al sistema central.
Aplicado a	RF-54
Restricción origen RF	RF-54 — Restricciones: *"Eliminación condicionada por confirmación: Los datos solo pueden eliminarse del buffer tras confirmación explícita de persistencia exitosa en el backend."* / *"No se permiten duplicados en el backend."*
Justificación	La coherencia entre es_sincronizado y fecha_sincronizacion debe ser garantizada. Si se marca como sincronizado, debe haber una fecha de sincronización.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_buffer_sincronizacion_coherencia()
RETURNS TRIGGER AS $$
BEGIN
    -- Si se marca como sincronizado, debe tener fecha de sincronización
    IF NEW.es_sincronizado = TRUE AND NEW.fecha_sincronizacion IS NULL
THEN
        RAISE EXCEPTION
            'Si es_sincronizado = true, fecha_sincronizacion es
obligatoria. (RF-54)'
            USING ERRCODE = 'P0001';
    END IF;
    -- Si se revierte el estado de sincronizado a no sincronizado, no
está permitido
    IF OLD.es_sincronizado = TRUE AND NEW.es_sincronizado = FALSE THEN
        RAISE EXCEPTION
            'No se puede revertir el estado de sincronización de TRUE a
FALSE. '
            'La sincronización es un estado final e irreversible.
(RF-54)'
            USING ERRCODE = 'P0001';
```

```
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_buffer_sincronizacion_coherencia
BEFORE UPDATE ON modulo3.buffers
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_buffer_sincronizacion_coherencia();
```

TRG-M03-16 — trg_eventos_edge_sla_automatico

Descripción	Calcula automáticamente el campo cumple_sla de cada evento de Edge Computing comparando la latencia_ms registrada contra el umbral de 500 ms definido en RF-55. Este cálculo se realiza en la BD para garantizar que el campo siempre sea consistente con la latencia real registrada, evitando situaciones donde el backend pueda marcar incorrectamente un evento como cumplidor del SLA cuando su latencia real lo excede. Si no hay latencia registrada, el campo queda como nulo indicando que el cumplimiento del SLA no pudo evaluarse.
Aplicado a	RF-55
Restricción origen RF	RF-55 — Descripción: *"El tiempo máximo de procesamiento por medición no debe exceder los 500 ms."* / CA-1: *"la detección de desviación debe completarse en ≤ 500 ms."*
Justificación	El campo cumple_sla debe calcularse automáticamente basado en latencia_ms. Evita inconsistencias si el backend omite calcularlo o lo calcula incorrectamente.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_eventos_edge_sla_automatico()
```

```
RETURNS TRIGGER AS $$
BEGIN
    -- Calcular automáticamente cumple_sla basado en latencia
    IF NEW.latencia_ms IS NOT NULL THEN
        NEW.cumple_sla := (NEW.latencia_ms <= 500);
    ELSE
        -- Si no hay latencia registrada, se considera no evaluable
        NEW.cumple_sla := NULL;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_eventos_edge_sla_automatico
BEFORE INSERT ON modulo3.eventos_edge_computing
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_eventos_edge_sla_automatico();
```

TRG-M03-17 — trg_eventos_edge_inmutable

Descripción	Impide cualquier modificación o eliminación sobre los eventos de Edge Computing ya persistidos. Estos eventos forman parte del historial técnico de procesamiento del módulo M03 y son consumidos por RF-63 como parte de la trazabilidad de auditoría del ecosistema IoT. Su inmutabilidad garantiza que la secuencia de eventos de detección de desviaciones, generación de alertas y procesamiento en el borde pueda ser reconstruida fielmente en cualquier momento posterior para diagnóstico de incidentes o auditorías regulatorias.
Aplicado a	RF-55
Restricción origen RF	RF-55 — Postcondiciones: *"Registro Local para Trazabilidad: El evento queda almacenado en el dispositivo Edge: logs técnicos, historial reciente."* / RF-63 — Restricciones: *"Los registros de auditoría*

	no pueden ser modificados ni eliminados."
Justificación	Los eventos de Edge Computing son parte del historial técnico y de auditoría. Su inmutabilidad garantiza la trazabilidad completa.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_eventos_edge_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            'Los eventos de Edge Computing son inmutables y no pueden
eliminarse. (RF-55 / RF-63)'
            USING ERRCODE = 'P0001';
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            'Los eventos de Edge Computing son inmutables y no pueden
modificarse. (RF-55 / RF-63)'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_eventos_edge_inmutable
BEFORE UPDATE OR DELETE ON modulo3.eventos_edge_computing
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_eventos_edge_inmutable();
```

TRG-M03-18 — trg_estados_conectividad_fecha_no_futura

Descripción	Valida que la fecha de registro de un estado de conectividad no sea posterior al momento actual del servidor. Los estados de conectividad representan el comportamiento histórico real de los dispositivos IoT en la

	red LoRaWAN. Un estado registrado con fecha futura es cronológicamente imposible y distorsionaría el historial de disponibilidad de la infraestructura IoT, afectando métricas como MTBF y MTTR calculadas en M08, y comprometiendo la interpretación correcta de los periodos sin datos en el historial de lecturas de RF-59.
Aplicado a	RF-60
Restricción origen RF	RF-60 — Proceso Fase 1: *"timestamp_heartbeat coherente (no futuro, sincronizado por NTP)."* / Restricciones: *"Sincronización de Tiempo: El dispositivo debe estar sincronizado."*
Justificación	Un estado de conectividad registrado con fecha futura es inválido por definición y viola la coherencia temporal del historial.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_estados_conectividad_fecha_no_futura()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_registro > NOW() THEN
        RAISE EXCEPTION
            'La fecha_registro del estado de conectividad [%] no puede
ser futura. (RF-60)',
            NEW.fecha_registro
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_estados_conectividad_fecha_no_futura
BEFORE INSERT ON modulo3.estados_conectividad
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_estados_conectividad_fecha_no_futura();
```

TRG-M03-19 — trg_estados_conectividad_inmutable

Descripción	Garantiza que ningún registro del historial de estados de conectividad de los dispositivos IoT pueda ser modificado o eliminado una vez persistido. RF-60 establece explícitamente que este historial es de tipo append-only, ya que constituye la evidencia técnica de cuándo y por cuánto tiempo cada dispositivo estuvo activo, sin señal, en modo buffer o en mantenimiento. Esta información es crítica para que RF-59 represente correctamente los vacíos temporales en el historial de lecturas y para que M06 pueda evaluar la cobertura del sistema de medición que sustenta la valoración NIC 41.
Aplicado a	RF-60
Restricción origen RF	RF-60 — Restricciones: *"Inmutabilidad del Historial de Estados: El historial de cambios de estado de cada dispositivo debe ser inmutable. Solo se pueden agregar nuevas entradas; no se permite modificar ni eliminar el registro de transiciones anteriores."*
Justificación	El historial de estados de conectividad es evidencia técnica auditada. Su inmutabilidad es un requisito explícito del requerimiento.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_estados_conectividad_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            'El historial de estados de conectividad es inmutable y no
puede eliminarse. (RF-60 Restricciones)'
            USING ERRCODE = 'P0001';
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            'El historial de estados de conectividad es inmutable y no
puede modificarse. (RF-60 Restricciones)'
            USING ERRCODE = 'P0001';
```

```
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_estados_conectividad_inmutable
BEFORE UPDATE OR DELETE ON modulo3.estados_conectividad
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_estados_conectividad_inmutable();
```

TRG-M03-20 — trg_transmisiones_intentos_positivos

Descripción	Valida que el número de intentos de transmisión sea un valor positivo y no supere el máximo de 5 reintentos definido en RF-56. Un valor negativo es inválido por definición. Superar el máximo de 5 reintentos indica que el paquete debió haber sido almacenado en el buffer local de RF-54 para sincronización diferida en lugar de continuar reintentando indefinidamente. Este trigger protege la integridad de las métricas de red y garantiza que la política de reintentos sea respetada a nivel de persistencia.
Aplicado a	RF-56
Restricción origen RF	RF-56 — Restricciones: *"Política de Reintentos: Número máximo de reintentos: 5."* / RF-54 — Restricciones: *"Soporte de múltiples frecuencias sin afectar integridad."*
Justificación	El número de intentos de transmisión debe ser un valor positivo y no puede ser negativo. Validación de dominio simple pero crítica para la integridad de métricas de red.

```
CREATE OR REPLACE FUNCTION
```

```
modulo3.fn_trg_transmisiones_intentos_positivos()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.intentos < 0 THEN
        RAISE EXCEPTION
            'El número de intentos de transmisión no puede ser negativo.
Valor: %. (RF-56)',
            NEW.intentos
            USING ERRCODE = 'P0001';
    END IF;
    IF NEW.intentos > 5 THEN
        RAISE EXCEPTION
            'El número de intentos de transmisión supera el máximo
permitido de 5. '
            'Valor: %. Si se superan 5 intentos, el paquete debe
almacenarse en buffer (RF-54). (RF-56 Restricciones)',
            NEW.intentos
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_transmisiones_intentos_positivos
BEFORE INSERT OR UPDATE ON modulo3.transmiciones_mqqt
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_transmisiones_intentos_positivos();
```

TRG-M03-21 — trg_transmisiones_fecha_no_futura

Descripción	Impide el registro de transmisiones MQTT con fecha posterior al momento actual del servidor. Una transmisión con timestamp futuro es físicamente imposible y representa una desincronización del reloj del gateway o del dispositivo. Dado que las transmisiones MQTT son el mecanismo de transporte de la telemetría desde el campo hasta el backend, la coherencia temporal de sus registros es fundamental para calcular latencias reales de la red LoRaWAN y diagnosticar

	problemas de conectividad de manera fiable.
Aplicado a	RF-56
Restricción origen RF	RF-56 — Restricciones: *"Consistencia Temporal Obligatoria: Los datos dentro del paquete deben: mantener orden cronológico, no contener timestamps futuros."*
Justificación	Una transmisión con fecha futura es incoherente cronológicamente.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_transmisiones_fecha_no_futura()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_transmision > NOW() THEN
        RAISE EXCEPTION
            'La fecha_transmision [%] no puede ser futura. (RF-56
Restricciones)',
            NEW.fecha_transmision
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_transmisiones_fecha_no_futura
BEFORE INSERT ON modulo3.transmiciones_mqqt
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_transmisiones_fecha_no_futura();
```

TRG-M03-22 — trg_lecturas_sensores_timestamp_no_futuro

Descripción	Valida tres condiciones temporales sobre cada lectura de sensor. Primero, que la fecha

	de captura no sea futura. Segundo, que la fecha de recepción en el backend no sea futura. Tercero y más importante, que la fecha de recepción no sea anterior a la fecha de captura, ya que es físicamente imposible que el sistema reciba un dato antes de que el sensor lo haya capturado. Esta tercera validación detecta inconsistencias causadas por desincronización severa de relojes entre el dispositivo y el servidor, que podrían pasar inadvertidas si solo se validaran las fechas de forma independiente.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Fase 3: *"timestamp_captura ≤ hora actual del servidor."* / RF-55.1: *"timestamp_captura: No puede ser futuro."*
Justificación	La tabla lecturas_sensores es una capa adicional al sistema. Sus timestamps también deben ser coherentes.

```
CREATE OR REPLACE FUNCTION
modulo3.fn_trg_lecturas_sensores_timestamp_no_futuro()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.fecha_captura > NOW() THEN
        RAISE EXCEPTION
            'La fecha_captura de la lectura del sensor [%] no puede ser
futura. (RF-53 Fase 3)',
            NEW.fecha_captura
            USING ERRCODE = 'P0001';
    END IF;
    IF NEW.fecha_recepcion > NOW() THEN
        RAISE EXCEPTION
            'La fecha_recepcion de la lectura del sensor [%] no puede
ser futura. (RF-53)',
            NEW.fecha_recepcion
            USING ERRCODE = 'P0001';
    END IF;
    -- La fecha_recepcion debe ser posterior o igual a fecha_captura
    IF NEW.fecha_recepcion < NEW.fecha_captura THEN
        RAISE EXCEPTION
            'La fecha_recepcion [%] no puede ser anterior a la
fecha_captura [%]. (RF-53)',
```

```
            NEW.fecha_recepcion, NEW.fecha_captura
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_lecturas_sensores_timestamp_no_futuro
BEFORE INSERT ON modulo3.lecturas_sensores
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_lecturas_sensores_timestamp_no_futuro();
```

TRG-M03-23 — trg_lecturas_sensores_inmutable

Descripción	Impide cualquier modificación o eliminación sobre las lecturas de sensores ya persistidas. Al igual que la tabla de telemetrías, las lecturas de sensores forman parte del historial inmutable de captura de datos del ecosistema IoT. Su integridad es necesaria para garantizar que el dato que se muestra en el monitoreo en tiempo real de RF-58, que se consulta en el historial de RF-59 y que se usa como insumo para las alertas de RF-57 sea siempre el mismo dato original capturado por el hardware, sin posibilidad de alteración posterior por ningún proceso o usuario del sistema.
Aplicado a	RF-53
Restricción origen RF	RF-53 — Restricciones: *"Inmutabilidad del Historial: Una vez que una medición ha sido almacenada y validada, el sistema no permitirá su modificación ni eliminación."*
Justificación	Las lecturas de sensores son igualmente parte del historial inmutable de telemetría.

```
CREATE OR REPLACE FUNCTION modulo3.fn_trg_lecturas_sensores_inmutable()
```

```
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            'Las lecturas de sensores son inmutables y no pueden
eliminarse. (RF-53 Restricciones)'
            USING ERRCODE = 'P0001';
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            'Las lecturas de sensores son inmutables y no pueden
modificarse. (RF-53 Restricciones)'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_lecturas_sensores_inmutable
BEFORE UPDATE OR DELETE ON modulo3.lecturas_sensores
FOR EACH ROW
EXECUTE FUNCTION modulo3.fn_trg_lecturas_sensores_inmutable();
```